



MACHINE LEARNING · CLASSIFICATION

Logistic Regression

From Linear Scores to Probabilities and Classes

Nancy Ahmed

What Is Logistic Regression?

Netflix wants to know: will a user subscribe? *Is that a number, or a category?*



Regression

Predicts a continuous value

- House price
- Temperature
- Salary



Classification

Predicts a category

- Spam or Not Spam
- Cancer or Healthy
- Pass or Fail

Why Is It Called “Regression”?

It still starts as a linear equation:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Exactly the same equation used in Linear Regression — the only difference is what happens next.

Linear Equation



Sigmoid



Probability



Class

Why Can't We Use Linear Regression?

Hours Studied	Pass
1	0
2	0
3	0
5	1

Suppose Linear Regression learns: $y = 0.3x - 0.4$

1 hour studied

$$y = -0.1$$



Probability can't be negative

8 hours studied

$$y = 2.0$$



Probability can't exceed 1

Linear Regression output ranges over $(-\infty, \infty)$ — but probabilities must satisfy $0 \leq P \leq 1$

The Sigmoid Function

We keep z from before, and squeeze it through:

$$\sigma(z) = 1 / (1 + e^{-z})$$

Any real number in — a probability between 0 and 1 out.

$z = 0$

0.500

50% chance

$z = 5$

0.993

Very high probability

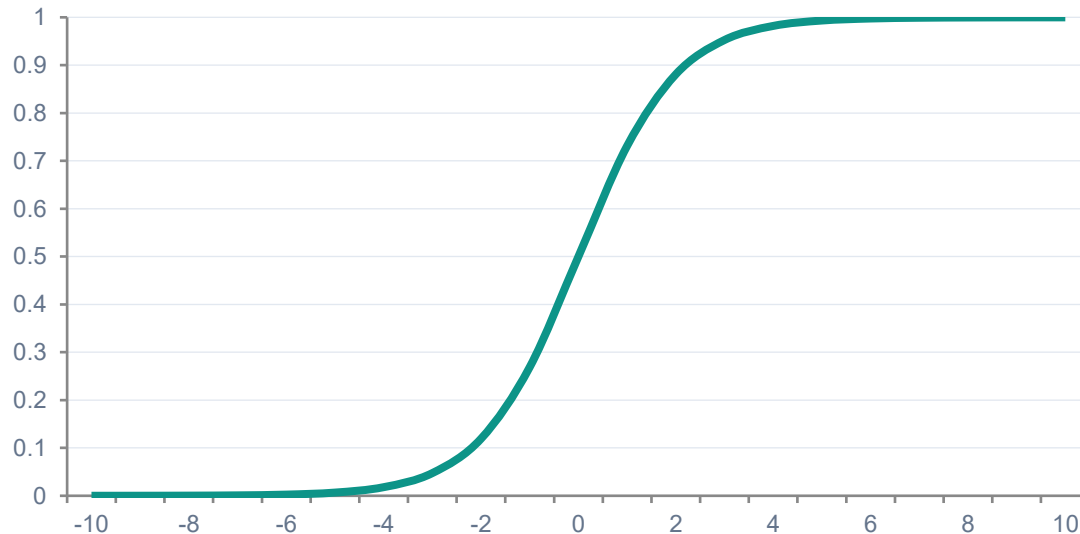
$z = -5$

0.0067

Very low probability

The Sigmoid Curve

z	Probability
-10	≈ 0
-2	0.12
0	0.50
2	0.88
10	≈ 1



Far left: almost zero. **Middle:** changes rapidly. **Far right:** almost one.

What Exactly Is z ?

Example — predicting admission from three features:



GPA



Study Hours

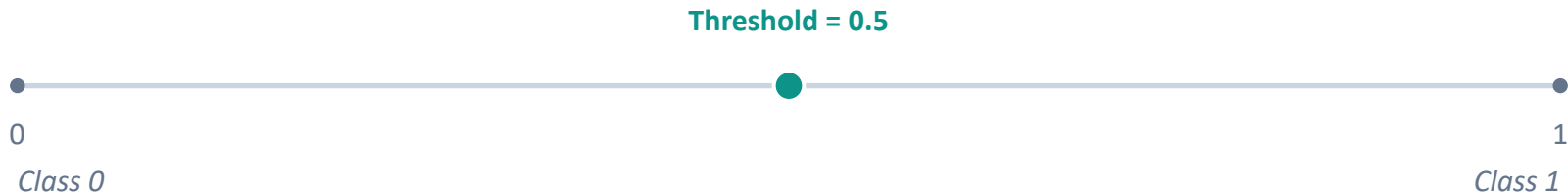


Attendance

$$z = w_1(\text{GPA}) + w_2(\text{Hours}) + w_3(\text{Attendance}) + b$$

z is just a raw score — it could come out as **-20, 5, 100, or -300**. Sigmoid is what turns that score into a probability.

From Probability to Class



If $P \geq 0.5 \rightarrow$ predict Class 1. Otherwise $\rightarrow$ predict Class 0.

Student A

$P = 0.82$

Pass



Student B

$P = 0.31$

Fail



Tuning the Decision Threshold

Threshold

0.3

High Recall

Low Precision

Almost everyone is classified positive.

Threshold

0.5

Balanced

The default

A neutral, commonly used starting point.

Threshold

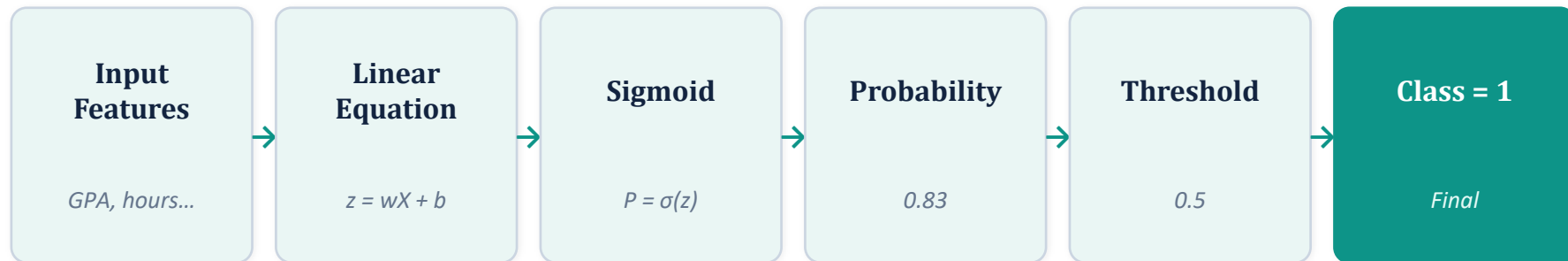
0.8

High Precision

Lower Recall

Only very confident cases count as positive.

The Complete Pipeline



One continuous pipeline: a linear score is reshaped into a probability, then collapsed into a decision.

KEY TAKEAWAYS

Logistic Regression, in Four Ideas

1

It predicts categories, not numbers — despite the name “Regression.”

2

The sigmoid function converts any real-valued score into a probability between 0 and 1.

3

A threshold — not just the model — makes the final call, and it can be tuned.

4

It shares the same linear backbone as Linear Regression, with one extra step.



MACHINE LEARNING · MODEL EVALUATION

Boundaries, Loss & Training

How Logistic Regression Decides, Learns, and Is Judged

From Point to Hyperplane

One feature — Hours Studied

Hours = 4.5

Fail

Pass

1 Feature

Point / threshold

2 Features

Line

3 Features

Plane

Many Features

Hyperplane

Why Not Mean Squared Error?

Mean Squared Error

$$MSE = (1/n) \sum (y - \hat{y})^2$$

- ✗ Combined with sigmoid, the loss surface becomes non-convex
- ✗ Gradient Descent can get stuck or converge unreliably

Binary Cross-Entropy

$$L = -(y \cdot \log(p) + (1-y) \cdot \log(1-p))$$

- ✓ Convex for Logistic Regression — one clear minimum
- ✓ Works reliably with Gradient Descent

Binary Cross-Entropy in Action

$$L = -(y \cdot \log(p) + (1-y) \cdot \log(1-p))$$

Good Prediction

Actual = 1 Predicted = 0.99

≈ 0.01

Confident and correct



Bad Prediction

Actual = 1 Predicted = 0.01

≈ 4.6

Confident but wrong



Gradient Descent

Gradient Descent is an optimization algorithm that automatically adjusts the model's weights to reduce the prediction error (loss).

$$W_{new} = W_{old} - \alpha \cdot \partial L / \partial w$$

Gradient is positive

The weight decreases

Gradient is negative

The weight increases

Learning rate α

Controls how large each step is

Training Process: Six Steps

1 Initialize Weights

Start random, e.g. $w=0.2$, $b=-0.5$

2 Make Predictions

$z = wX + b$, then Sigmoid $\rightarrow$ probability

3 Compute the Loss

Compare probability vs. actual label (BCE)

4 Compute Gradients

Find how loss changes w.r.t. each weight

5 Update Weights

$w = w - \alpha \cdot \partial L / \partial w$

6 Repeat

Thousands of times until loss stops improving

The Accuracy Trap

100 patients — 99 healthy, 1 sick. The model predicts everyone is healthy.

Accuracy

99%

Looks great... at first



It missed the only sick patient

Accuracy alone can be misleading with imbalanced datasets.

This is exactly why we need better metrics.

The Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

TP — True Positive

Predicted positive, actually positive. Correct.

TN — True Negative

Predicted negative, actually negative. Correct.

FP — False Positive

Predicted positive, actually negative. False alarm.

FN — False Negative

Predicted negative, actually positive. Missed case.

Precision, Recall & F1-Score

Precision

$$TP / (TP + FP)$$

When we predict Positive, how often are we right?

Costly false alarms — e.g. spam filtering

Recall

$$TP / (TP + FN)$$

Of all actual positives, how many did we find?

Costly misses — e.g. cancer detection

F1-Score

$$2 \cdot P \cdot R / (P + R)$$

A single score balancing both.

Useful when both matter

The Accuracy Trap

100 patients — 90 healthy, 10 sick

Accuracy

93%

Looks great... at first

**The model predicts:
8 sick patients correctly.
Misses 2 sick patients.
Incorrectly labels 5
healthy patients as sick.**

The Confusion Matrix

	Predicted Positive "sick"	Predicted Negative "healthy"
Actual Positive "sick"	TP=8	FN=2
Actual Negative "healthy"	FP=5	TN=85

The model predicts:
8 sick patients correctly.
Misses 2 sick patients.
Incorrectly labels 5
healthy patients as sick.

Precision, Recall & F1-Score

Precision

$$\begin{aligned} & TP / (TP + FP) \\ & = 8 / (8 + 5) = 61.5\% \end{aligned}$$

When we predict Positive, how often are we right?

Costly false alarms — e.g. spam filtering

Recall

$$\begin{aligned} & TP / (TP + FN) \\ & = 8 / (8 + 2) = 80\% \end{aligned}$$

Of all actual positives, how many did we find?

Costly misses — e.g. cancer detection

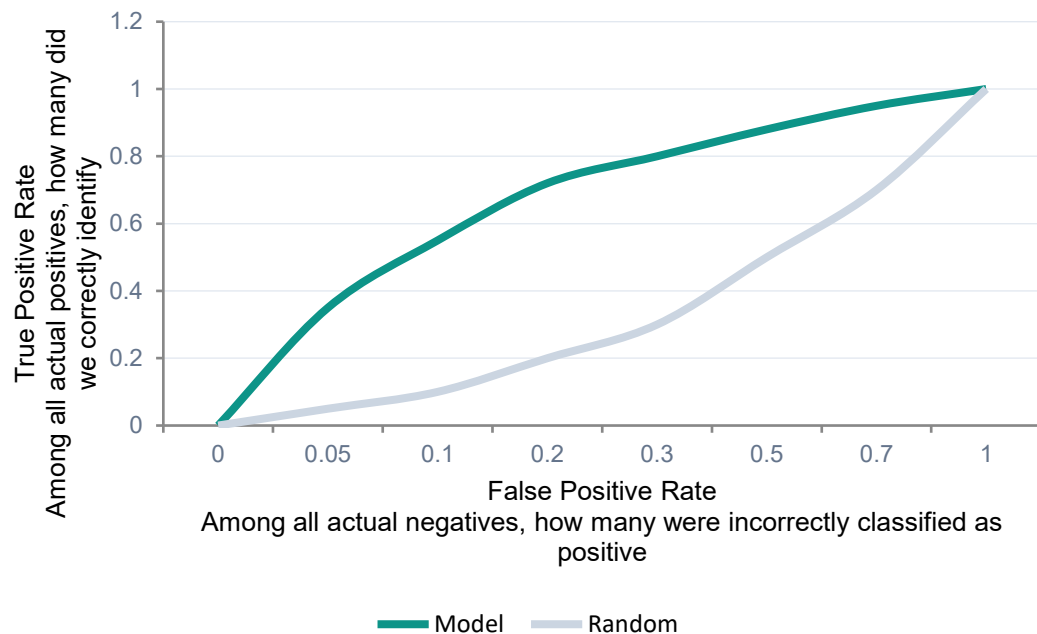
F1-Score

$$\begin{aligned} & 2 \cdot P \cdot R / (P + R) \\ & = 69.5\% \end{aligned}$$

A single score balancing both.

Useful when both matter

ROC Curve & AUC

**0.5**

Random guessing

0.7 – 0.8

Acceptable

0.8 – 0.9

Good

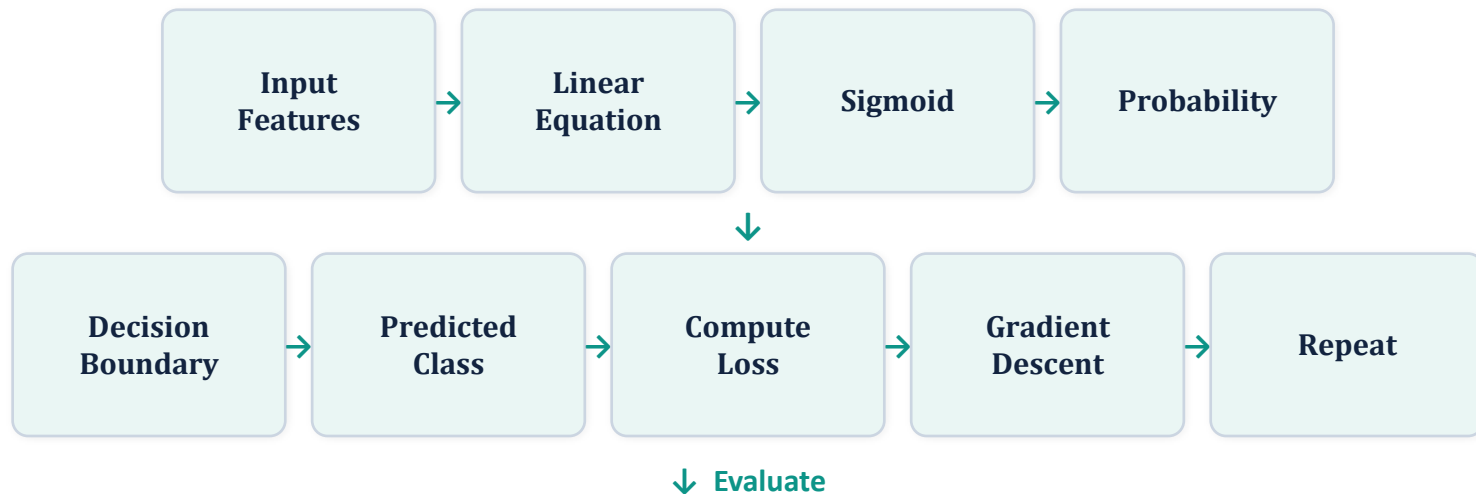
0.9 – 1.0

Excellent

1.0

Perfect classifier

The Complete Workflow



Confusion Matrix

Accuracy

Precision

Recall

F1-Score

ROC / AUC

KEY TAKEAWAYS

Boundaries, Loss & Evaluation

- 1 The decision boundary is simply where $z = 0$ — a point, line, plane, or hyperplane.
- 2 Binary Cross-Entropy penalizes confident, wrong predictions far more than uncertain ones.
- 3 Training repeats one loop: predict $\rightarrow$ compute loss $\rightarrow$ compute gradients $\rightarrow$ update weights.
- 4 Accuracy alone can mislead on imbalanced data — pair it with Precision, Recall, F1, and ROC/AUC.

Assumptions of Logistic Regression

Every algorithm makes assumptions about the data. When they roughly hold, the model performs well — when violated, performance can suffer.

- 1 Binary Target** — The target variable has exactly two classes (e.g. Spam / Not Spam).
- 2 Independent Observations** — Each training example stands on its own — no repeated or linked records.
- 3 Little Multicollinearity** — Features shouldn't carry almost the same information (e.g. height in cm and meters).
- 4 No Extreme Outliers** — Unusual values can pull the decision boundary away from the true pattern.

Binary Target & Independent Observations



Binary Target

The model is designed for two classes.

- Spam / Not Spam
- Pass / Fail
- Disease / Healthy

3+ classes? Extend with One-vs-Rest or Multinomial Logistic Regression.



Independent Observations

Each example should stand alone.

Good — Student A, Student B, Student C: each independent.

Bad — recording the same student's score 20 times.

Each customer = one observation, not repeated purchase copies.

Little Multicollinearity

What happens if two features carry almost the same information?

Height (cm)

≈ almost identical to ≈

Height (meters)

Age

≈ carries similar info as ≈

Year of Birth

When features are highly correlated, the model struggles to decide how much importance to assign to each one — coefficients become unstable and harder to interpret.

No Extreme Outliers

Outliers are unusual observations that can pull the decision boundary off course.

\$30,000

Typical salary



\$40,000

Typical salary



\$45,000

Typical salary



\$50,000

Typical salary



\$5,000,000

Extreme outlier — distorts the fitted boundary



One-vs-Rest (OvR)

A single sigmoid only outputs one probability, so classifying 3+ classes needs a strategy. OvR trains one binary model per class.

M1 Setosa vs. Not Setosa

M2 Versicolor vs. Not Versicolor

M3 Virginica vs. Not Virginica

Model	Probability
Setosa	0.10
Versicolor	0.80
Virginica	0.15



Prediction: Versicolor — highest probability wins.

Multinomial Logistic Regression (Softmax)

THE SOFTMAX FUNCTION

$$P(y = i) = e^{z_i} \div \sum_j e^{z_j}$$

Softmax computes a probability for every class at once — all outputs sum to exactly 1.

K = number of classes | z_i = score for class i

Class	Probability
Setosa	0.05
Versicolor	0.20
Virginica	0.75



Virginica has the highest probability — that's the prediction.

scikit-learn's `LogisticRegression()` handles OvR or multinomial automatically, depending on the solver.

Advantages



Simple & Fast

Easy to implement and computationally efficient, even on large datasets.



Easy to Interpret

Each coefficient shows how a feature affects the prediction — valued in healthcare and finance.



Produces Probabilities

Outputs a probability (e.g. 0.93), not just a class — letting you tune the decision threshold.



Works on Small Datasets

Can perform well with limited data, provided relationships aren't too complex.

Disadvantages

Linear Decision Boundary



Straight line — easy case

Learns a straight line (or hyperplane). Struggles when classes need a curved boundary to separate.

Sensitive to Outliers



One extreme point can shift the whole boundary

Large outliers can pull the estimated decision boundary away from the true underlying pattern.

Limited on Nonlinear Data



Interleaved — needs a curve

Struggles with images, speech, and complex data. Trees, SVMs with kernels, or neural nets often win here.

Real-World Applications



Disease Prediction

Positive / Negative



Spam Detection

Spam / Not Spam



Customer Churn

Will they leave?



Loan Approval

Approve / Reject



Credit Risk

Low / High Risk



Fraud Detection

Fraud / Legitimate



Sentiment Analysis

Positive / Negative

Python Example (scikit-learn)

```
from sklearn.linear_model import  
LogisticRegression  
  
model = LogisticRegression()  
model.fit(X_train, y_train)  
  
pred = model.predict(X_test)  
prob = model.predict_proba(X_test)
```

predict(X)

Returns the final class: [1, 0, 1, 1, 0] (1 = Positive, 0 = Negative)

predict_proba(X)

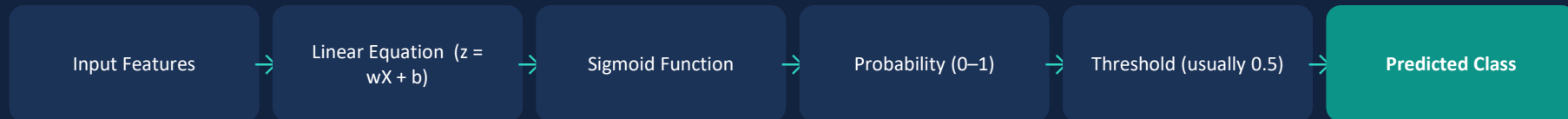
Returns probability per class:

```
[[0.10, 0.90],  
 [0.85, 0.15],  
 [0.30, 0.70]]
```

Row 1: $0.90 > 0.10 \rightarrow$ predicted class is 1.

PUTTING IT ALL TOGETHER

From Features to Prediction



Evaluate the result with:



Logistic Regression — simple, fast, interpretable, and still a strong first baseline.